

Data Structure: 스택

Prof. Jungmin Kwon
Spring, 2026

- 순차 자료구조의 개념
 - 구현할 자료들을 논리적 순서로 메모리에 연속 저장하는 구현 방식
 - 논리적인 순서와 물리적인 순서가 항상 일치해야 함

리스트 이름 = (원소 1, 원소 2, ..., 원소 n)
①

(a) 리스트 표현 형식

그림 3-1 리스트 표현 형식과 예



그림 3-3 선형 리스트의 메모리 저장 구조 예

동창 = (상원, 상범, 수영, 현정)

(b) 리스트 표현 예

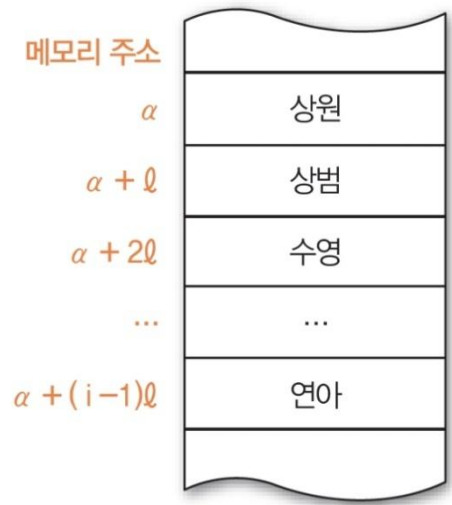


그림 3-4 선형 리스트에서 원소의 위치

■ 행렬



그림 3-15 행렬 A의 2차원 배열 표현 예

■ 다항식

$$P(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x^1 + a_0 x^0$$

그림 3-18 n차 다항식 P(x)

	[0]	[1]	[2]	[3]	...	[997]	[998]	[999]	[1000]
B	3	0	0	0	...	0	0	1	4

(a) 1차원 배열을 이용한 순차 자료구조

	[0]	[1]	
[0]	1000	3	→ $3x^{1000}$
[1]	1	1	→ x
[2]	0	4	→ 4

(b) 2차원 배열을 이용한 순차 자료구조

그림 3-20 희소 다항식의 순차 자료구조 표현 예 : $B(x) = 3x^{1000} + x + 4$

- 선형 리스트에서 원소 삽입과 알고리즘
 - 선형리스트 중간에 원소가 삽입되면, 그 이후의 원소들은 한 자리씩 자리를 뒤로 이동하여 물리적 순서를 논리적 순서와 일치시킴

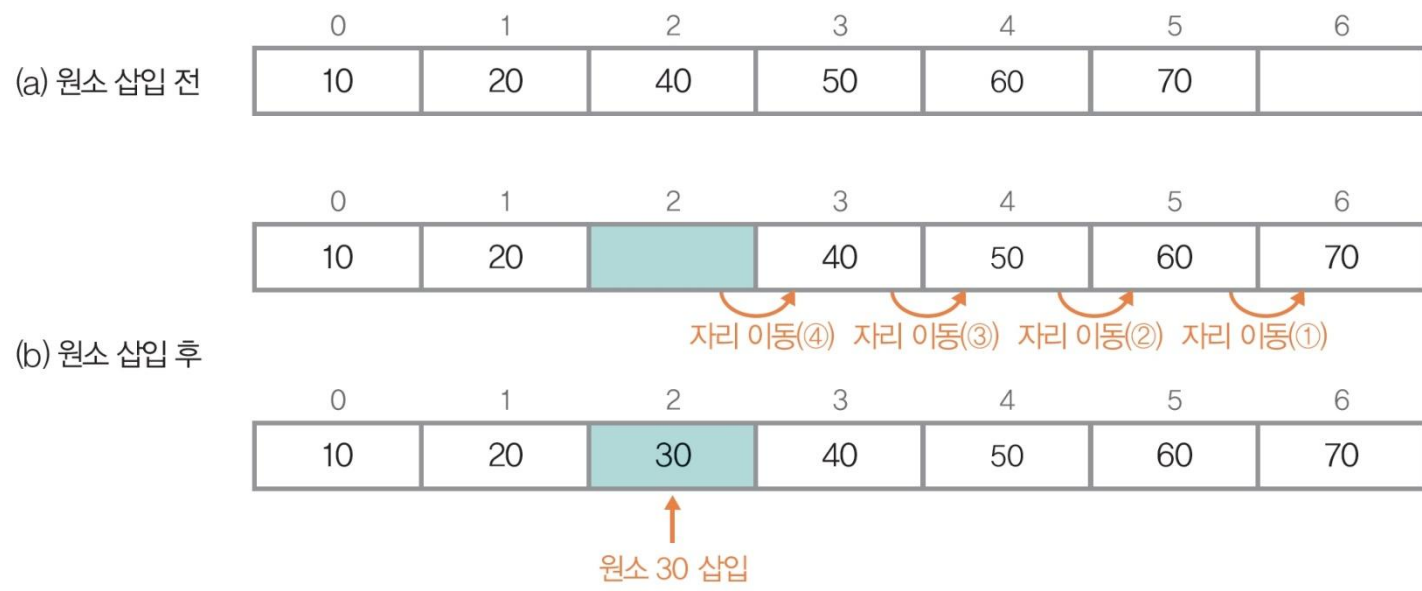
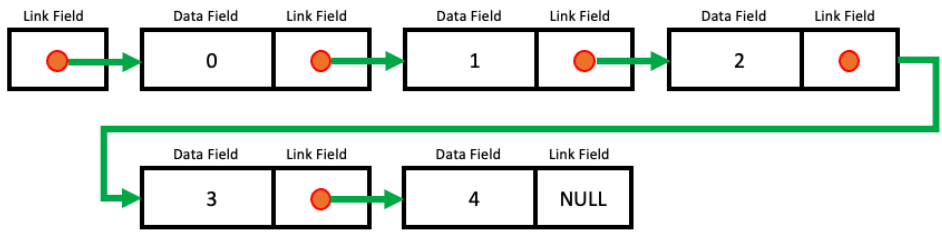


그림 3-11 선형 리스트에서의 원소 삽입

- 연결 자료구조 Linked Data Structure
 - 자료의 논리적인 순서와 물리적인 순서가 불일치
 - 각 원소에 저장되어 있는 다음 원소의 주소에 의해 순서가 연결되는 방식



<논리적 구조>

start	100	Link Field
100번지	0	Data Field
	160	Link Field
120번지	2	Data Field
	320	Link Field
160번지	1	Data Field
	120	Link Field
300번지	4	Data Field
	NULL	Link Field
320번지	3	Data Field
	300	Link Field

<물리적 구조>

■ 단순 연결 리스트에 삽입하는 방법

- ① 삽입할 노드를 준비한다.
- ② 새 노드의 데이터 필드에 값을 저장한다.
- ③ 새 노드의 링크값을 지정한다.
- ④ 리스트의 앞 노드에 새 노드를 연결한다.

그림 4-11 단순 연결 리스트에 노드를 삽입하는 방법

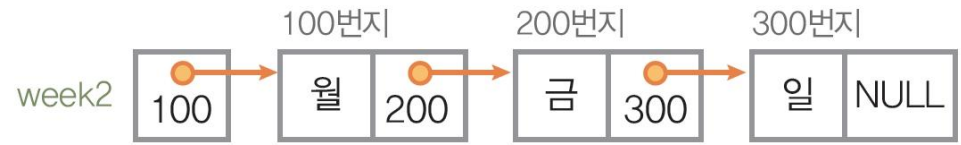
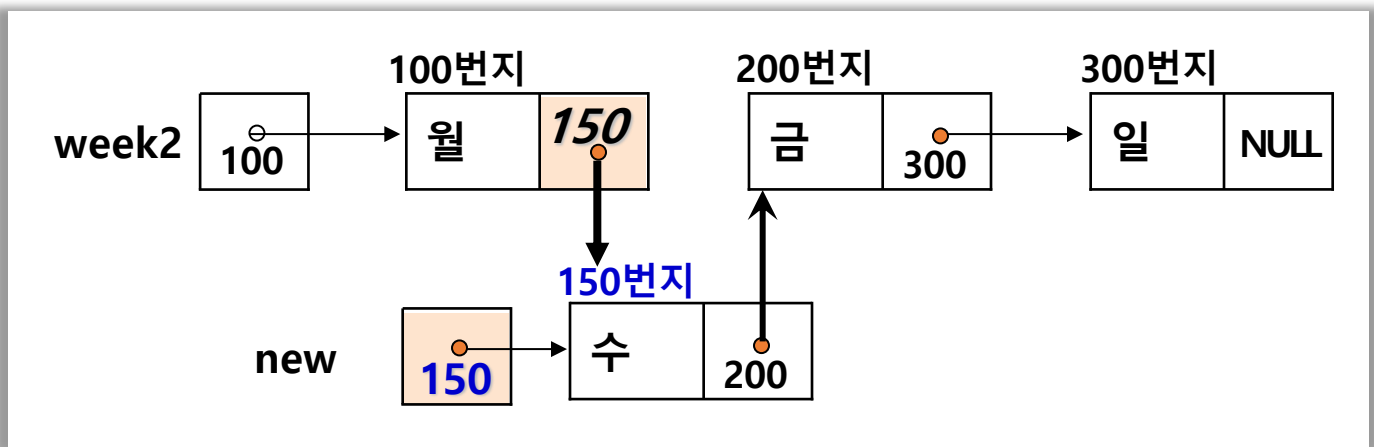


그림 4-12 단순 연결 리스트 week2에 노드를 삽입하기 전인 초기 상태



- 정렬된 리스트 유지하기
 - 다음과 같은 오름차순 정렬된 정수 리스트가 있다.
 - 이 리스트에 새로운 값을 삽입하되, 항상 정렬 상태를 유지하도록 프로그램을 작성하시오.
- 조건
 - 배열을 사용한 순차 리스트로 구현한다.
 - 최대 크기는 100이다.
 - 새로운 값은 적절한 위치에 삽입되어야 한다.
 - 기존 원소들은 필요에 따라 이동시켜야 한다.

```
C:\Users\Wjmkwo\Dropbox\Wjmkwo\...
현재 리스트: 1 3 5 7
삽입할 값 입력:
```

```
Microsoft Visual Studio 디버그
현재 리스트: 1 3 5 7
삽입할 값 입력: 6
결과 리스트: 1 3 5 6 7
```

■ 알고리즘 흐름

- 1. 새로운 숫자를 삽입할 위치 찾기
- 2. 새로운 숫자가 들어갈 뒤에 숫자들을 모두 한 칸씩 밀기
- 3. 새로운 값을 빈 자리에 삽입하기
- 4. 전체 리스트 길이 업데이트하기



그림 3-11 선형 리스트에서의 원소 삽입

Review) 연결 리스트 예제

- 정렬된 단일 연결 리스트에 삽입하기
 - 다음과 같은 오름차순 정렬된 단일 연결 리스트가 있다.
 - 이 리스트에 새로운 값을 삽입하되, 정렬 상태를 유지하도록 프로그램을 작성하시오.
- 조건
 - 단일 연결 리스트(singly linked list)를 사용한다.
 - 새로운 노드를 생성하여 삽입한다.
 - 항상 오름차순을 유지해야 한다.
 - NULL값 입력은 받지 않는다.

```

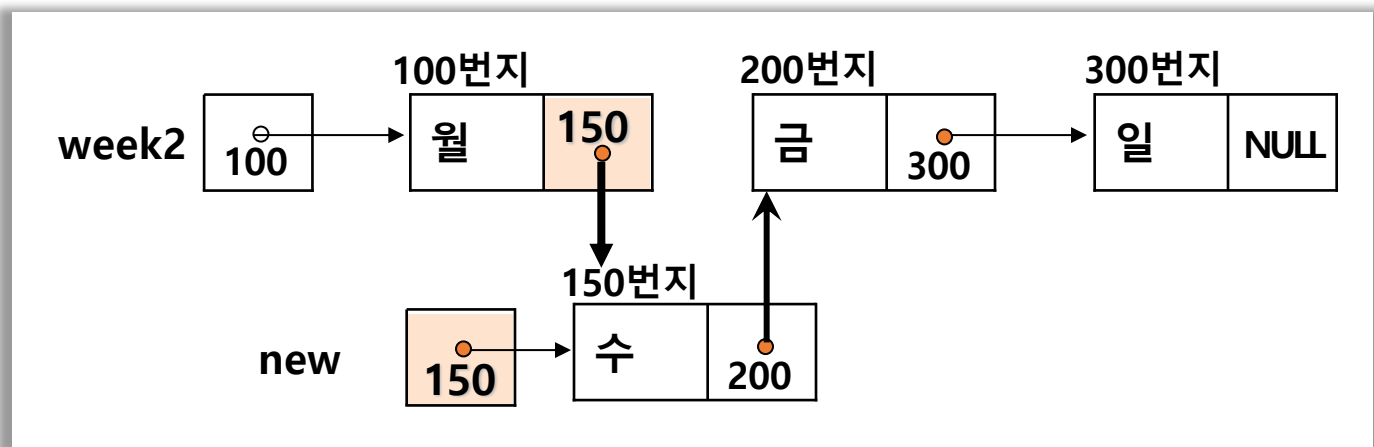
C:\Users\Wjmkwo\OneDrive\W
현재 리스트: 1 -> 3 -> 5 -> 7 -> NULL
삽입할 값 입력: |
    
```

```

Microsoft Visual Studio 디버그
현재 리스트: 1 -> 3 -> 5 -> 7 -> NULL
삽입할 값 입력: 6
결과 리스트: 1 -> 3 -> 5 -> 6 -> 7 -> NULL
    
```

■ 알고리즘 흐름

1. 노드 구조체 만들기
2. 새로운 노드 생성하는 함수 만들기
3. 노드를 정렬에 맞추어 삽입하는 함수 만들기
4. 메인 함수에서 새로운 값을 입력 받아 정렬에 맞추어 삽입하는 함수 사용하기



■ 학습목표

- 스택 자료구조에 대해 이해한다.
- 스택의 특징과 연산 방법을 알아본다.
- 순차 자료구조와 연결 자료구조를 이용해 스택을 구현해 본다.
- 스택을 응용하는 방법을 알아본다.

■ 내용

- 01스택의 이해
- 02 스택의 구현
- 03 스택의 응용

1. 스택의 이해 : 스택의 개념과 구조

- 스택(stack)
 - 접시를 쌓듯이 자료를 차곡차곡 쌓아 올린 형태의 자료구조



그림 5-1 스택의 개념 예



1. 스택의 이해 : 스택의 개념과 구조

■ 스택(stack)

- 스택에 저장된 원소는 top으로 정한 곳에서만 접근 가능
 - top의 위치에서만 원소를 삽입하므로, 먼저 삽입한 원소는 밑에 쌓이고, 나중에 삽입한 원소는 위에 쌓이는 구조
 - 마지막에 삽입(**L**ast-**I**n)한 원소는 맨 위에 쌓여 있다가 가장 먼저 삭제(**F**irst-**O**ut)됨 ⇨ **후입선출 구조 (LIFO, Last-In-First-Out)**

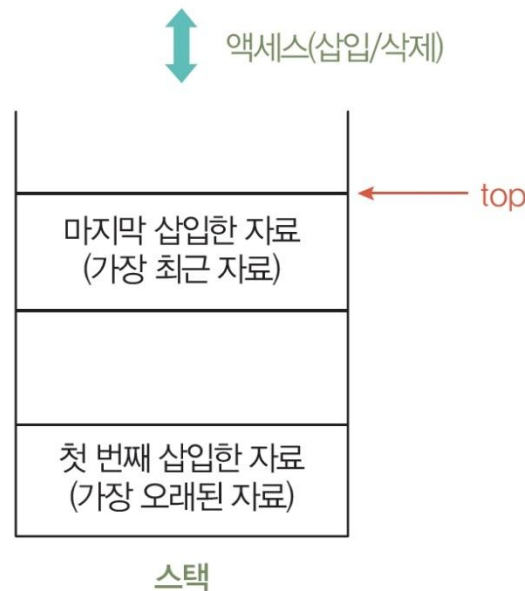


그림 5-2 스택의 구조

1. 스택의 이해 : 스택의 개념과 구조

- 후입선출 구조의 예1 : 연탄 아궁이

- 연탄을 하나씩 쌓으면서 아궁이에 넣으므로 마지막에 넣은 3번 연탄이 가장 위에 쌓여 있다.
- 연탄을 아궁이에서 꺼낼 때에는 위에서부터 하나씩 꺼내야 하므로 마지막에 넣은 3번 연탄을 가장 먼저 꺼내게 된다.

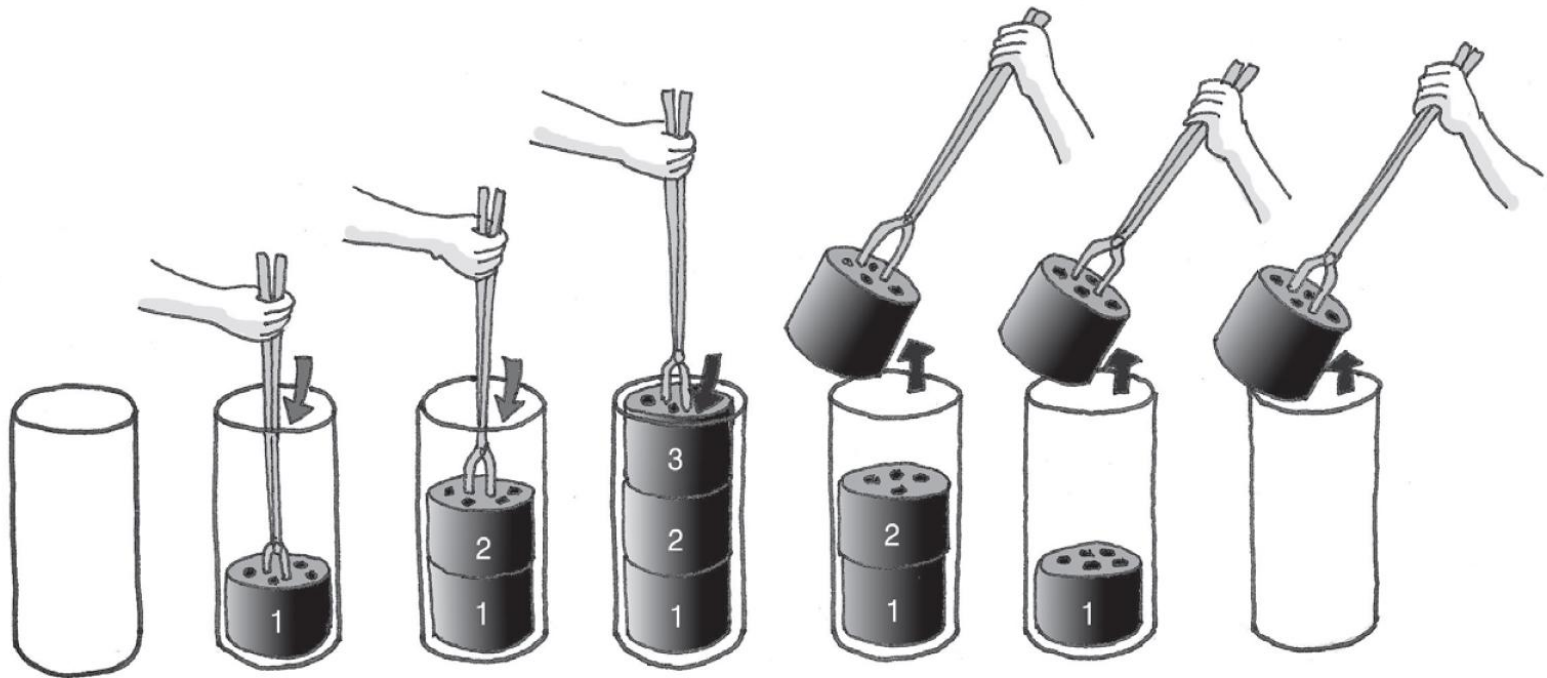


그림 5-3 스택의 LIFO 구조 예 : 연탄 아궁이

1. 스택의 이해 : 스택의 개념과 구조

- 후입선출 구조의 예2 : 슈퍼맨의 옷 갈아입기

- 슈퍼맨이 옷을 벗는 순서
 - ①장화 → ②망토 → ③빨간팬츠 → ④파란옷
- 슈퍼맨이 옷을 입는 순서
 - ④파란옷 → ③빨간팬츠 → ②망토 → ①장화

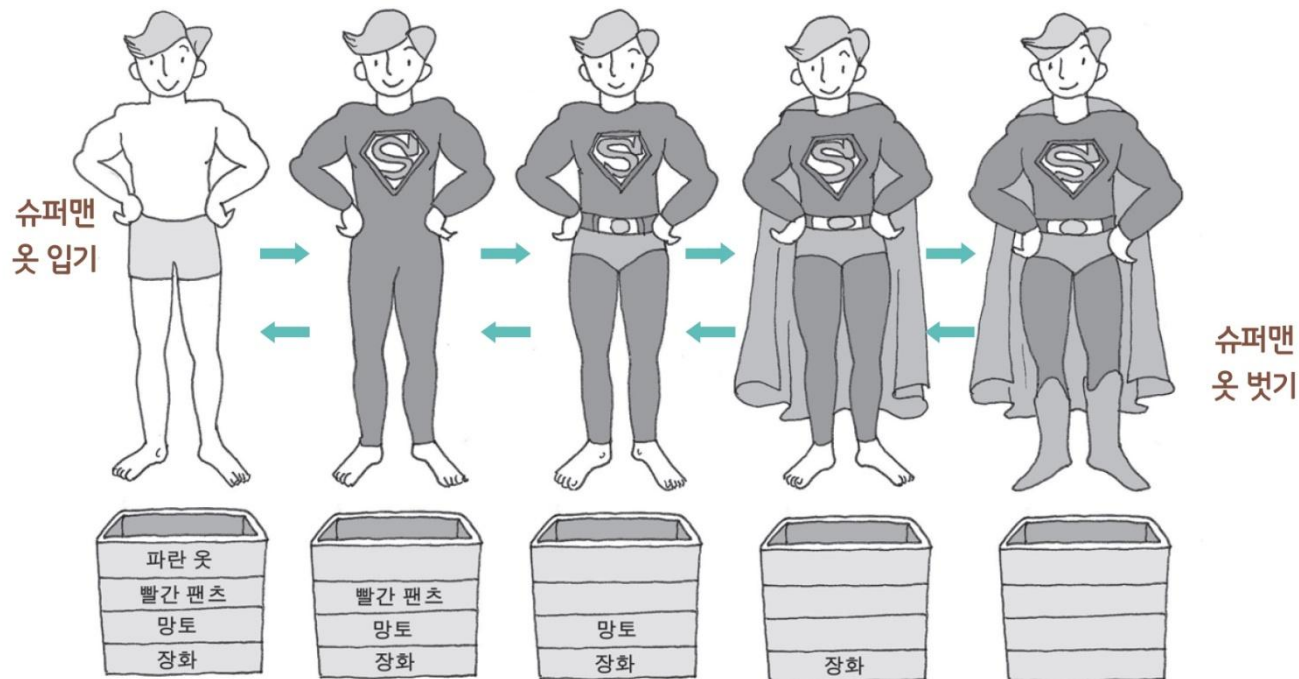


그림 5-4 스택의 LIFO 구조 예 : 슈퍼맨 옷 입기

1. 스택의 이해 : 스택의 개념과 구조

- 스택의 연산
 - 스택에서의 삽입 연산 : push
 - 스택에서의 삭제 연산 : pop

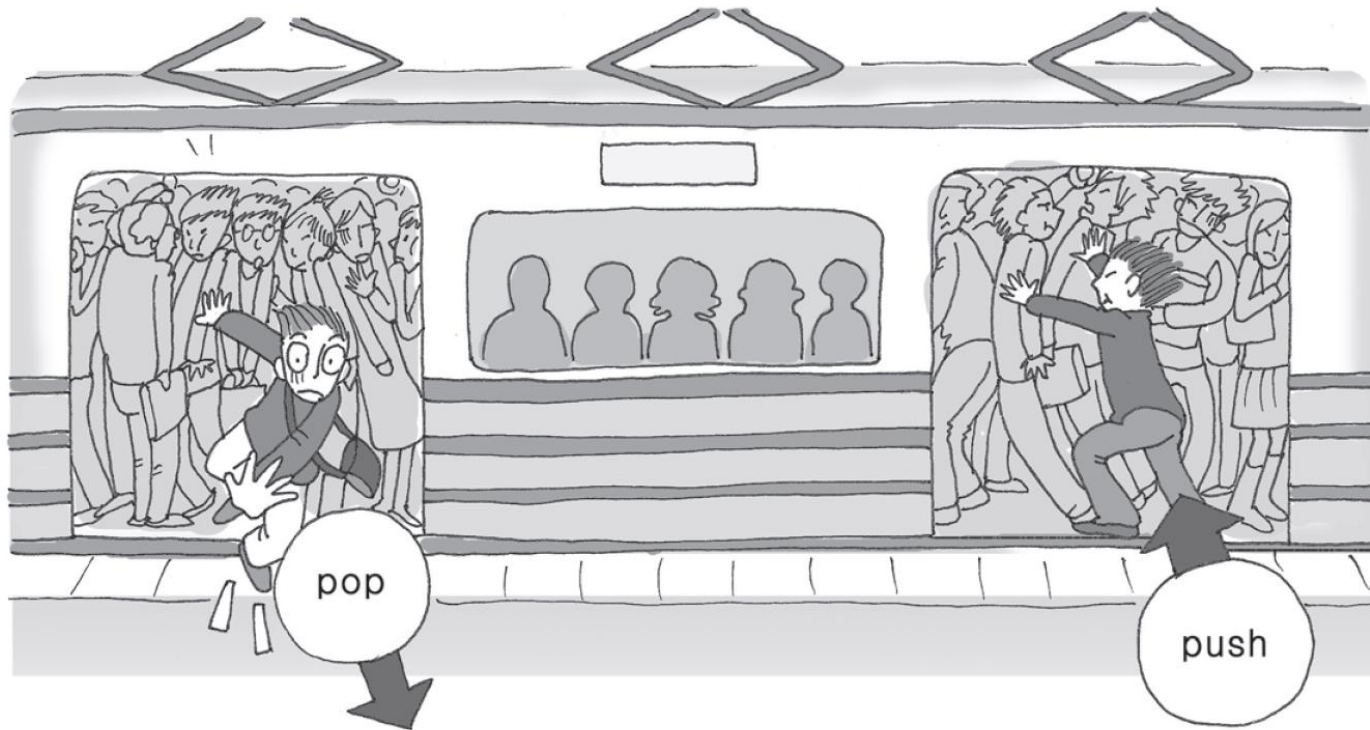


그림 5-5 만원 지하철에서의 pop과 push

1. 스택의 이해 : 스택의 개념과 구조

- 스택에서의 원소 삽입/삭제 과정

- [그림 5-6] 공백 스택에 원소 A, B, C를 순서대로 삽입하고 한번 삭제하는 연산과정 동안의 스택 변화

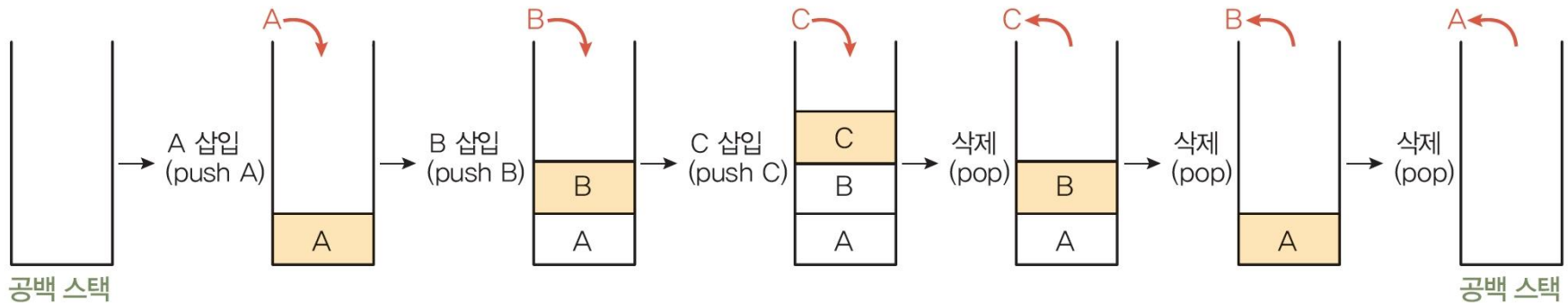


그림 5-6 스택의 데이터 삽입(push)과 삭제(pop) 과정

1. 스택의 이해 : 스택의 추상 자료형

ADT 5-1 스택의 추상 자료형

ADT Stack

데이터 : 0개 이상의 원소를 가진 유한 순서 리스트

연산 :

$S \in \text{Stack}; \text{item} \in \text{Element};$

// 공백 스택 S를 생성하는 연산

`createStack(S) ::= create an empty Stack S;`

// 스택 S가 공백인지 확인하는 연산

`isEmpty(S) ::= if (S is empty) then return true
 else return false;`

// 스택 S의 top에 item(원소)을 삽입하는 연산

`push(S, item) ::= insert item onto the top of Stack S;`

// 스택 S의 top에 있는 item(원소)을 삭제하는 연산

`pop(S) ::= if (isEmpty(S)) then return error
 else delete and return the top item of Stack S;`

// 스택 S의 top에 있는 item(원소)을 반환하는 연산

`peek(S) ::= if (isEmpty(S)) then return error
 else return the top item of the Stack S;`

End Stack

1. 스택의 이해 : 스택의 추상 자료형

■ 스택의 push() 알고리즘

① $top \leftarrow top + 1;$

- 스택 S에서 top이 마지막 자료를 가리키고 있으므로 그 위에 자료를 삽입하려면 먼저 top의 위치를 하나 증가
- 만약 이때 top의 위치가 스택의 크기(stack_SIZE)보다 크다면 오버플로우(overflow)상태가 되므로 삽입 연산을 수행하지 못하고 연산 종료

② $S(top) \leftarrow x;$

- 오버플로우 상태가 아니라면 스택의 top이 가리키는 위치에 x 삽입

알고리즘 5-1 스택의 원소 삽입

```
push(S, x)
  ① top ← top + 1;
  if (top > stack_SIZE) then overflow;
  else
    ② S(top) ← x;
  end push()
```

1. 스택의 이해 : 스택의 추상 자료형

■ 스택의 pop() 알고리즘

① return S(top);

- 공백 스택이 아니면 top에 있는 원소를 삭제하고 반환

② $top \leftarrow top - 1$;

- 스택의 마지막 원소가 삭제되면 그 아래 원소, 즉 스택에 남아 있는 원소 중에서 가장 위에 있는 원소가 top이 되어야 하므로 top 위치를 하나 감소

알고리즘 5-2 스택의 원소 삭제

```
pop(S)
  if (isEmpty(S)) then underflow;
  else {
    ① return S(top);
    ② top ← top - 1;
  }
end pop()
```

2. 스택의 구현: 순차 자료구조를 이용한 스택 구현

- 순차 자료구조를 이용한 스택의 구현
 - 순차 자료구조인 1차원 배열을 이용하여 구현
 - 스택의 크기 : 배열의 크기
 - 스택에 저장된 원소의 순서 : 배열 원소의 인덱스
 - 인덱스 0번 : 스택의 첫번째 원소
 - 인덱스 n-1번 : 스택의 n번째 원소
 - 변수 top : 스택에 저장된 마지막 원소에 대한 인덱스 저장
 - 공백 상태 : top = -1 (초기값)
 - 포화 상태 : top = n-1

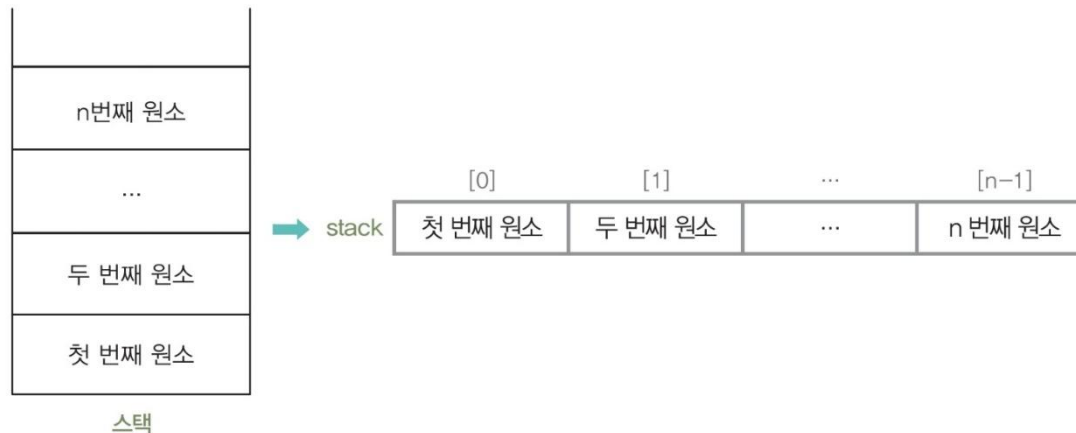
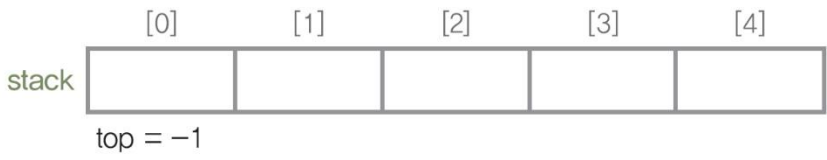


그림 5-7 1차원 배열을 이용한 순차 스택

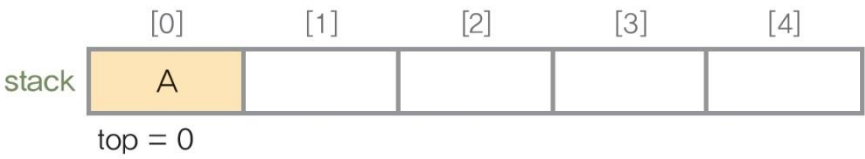
2. 스택의 구현: 순차 자료구조를 이용한 스택 구현

- 크기가 5인 스택을 생성하여 원소 A, B, C를 순서대로 삽입한 후에 원소 하나를 삭제하는 과정

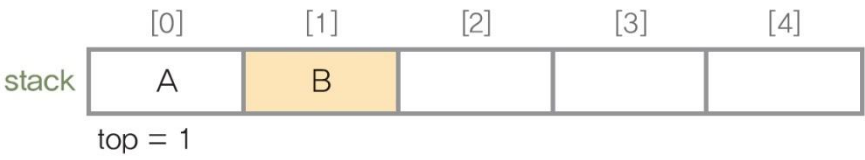
1 공백 스택 생성 : createStack(stack, 5);



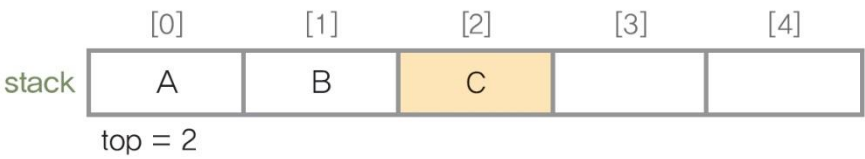
2 원소 A 삽입 : push(stack, A);



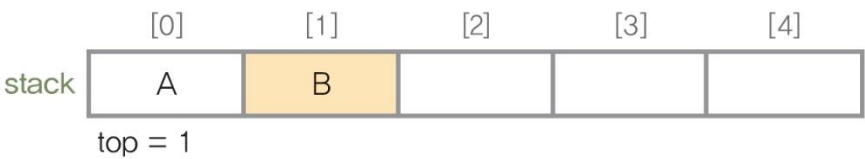
3 원소 B 삽입 : push(stack, B);



4 원소 C 삽입 : push(stack, C);



5 원소 삭제 : pop(stack);



2. 스택의 구현: 순차 자료구조를 이용한 스택 구현

- [예제 5-1] 순차 자료구조를 이용해 순차 스택 구현하기 : [교재 247p](#)
- 실행 결과

**** 순차 스택 연산 ****

```

STACK [ ]
STACK [ 1 ]
STACK [ 1 2 ]
STACK [ 1 2 3 ]
STACK [ 1 2 3 ] peek => 3
STACK [ 1 2 ]   pop  => 3
STACK [ 1 ]     pop  => 2
STACK [ ]       pop  => 1
    
```

2. 스택의 구현: 순차 자료구조를 이용한 스택 구현

- 순차 자료구조로 구현한 스택의 장점
 - 순차 자료구조인 1차원 배열을 사용하여 쉽게 구현
 - 순차 자료구조로 구현한 스택의 단점
 - 물리적으로 크기가 고정된 배열을 사용하므로 스택의 크기 변경 어려움
 - 순차 자료구조의 단점을 가짐
- 연결자료구조를 이용한 스택을 사용하면 크기 제한이 없는 스택을 만들 수 있다.

- 연결 자료구조를 이용한 스택의 구현
 - 단순 연결 리스트를 이용하여 구현
 - 스택의 원소 : 단순 연결 리스트의 노드
 - 스택 원소의 순서 : 노드의 링크 포인터로 연결
 - push : 리스트의 마지막에 노드 삽입
 - pop : 리스트의 마지막 노드 삭제
 - 변수 top : 단순 연결 리스트의 마지막 노드를 가리키는 포인터 변수
 - 초기 상태 : top = null

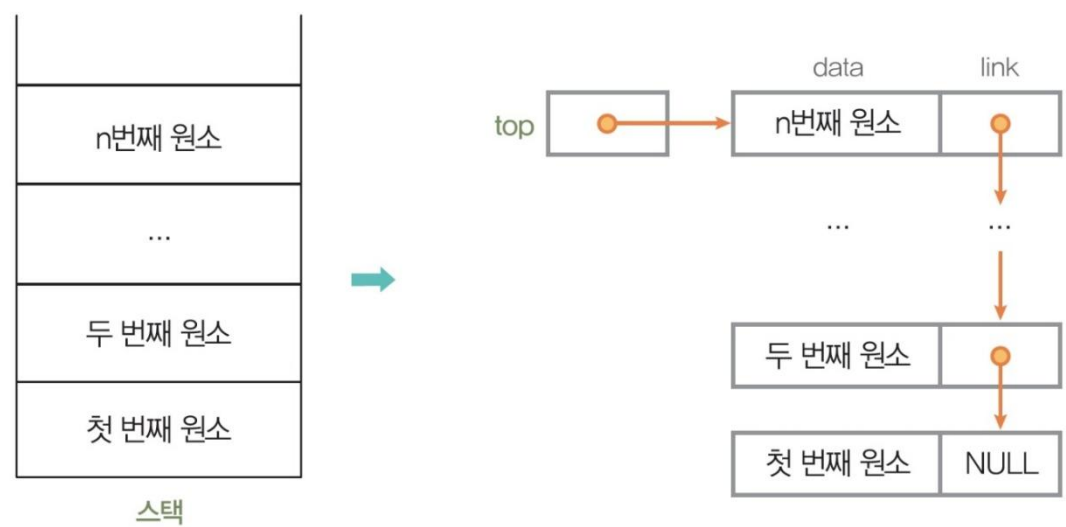


그림 5-8 단순 연결 리스트를 이용한 연결 스택

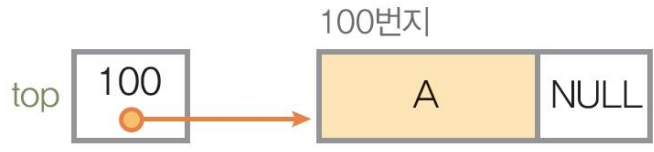
2. 스택의 구현:연결 자료구조를 이용한 스택의 구현

- 원소 A, B, C를 순서대로 삽입하면서 스택을 생성한 후에 원소 하나를 삭제하는 과정

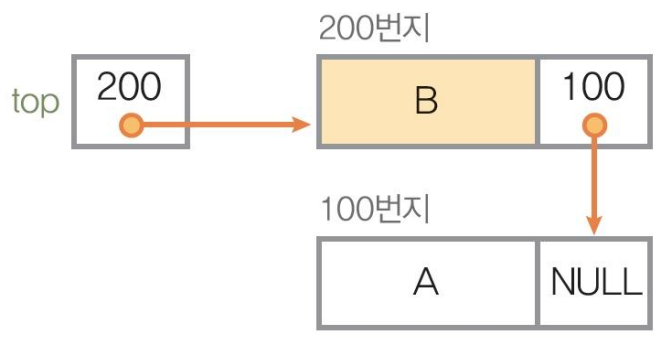
1 공백 스택 생성 : createStack(stack);



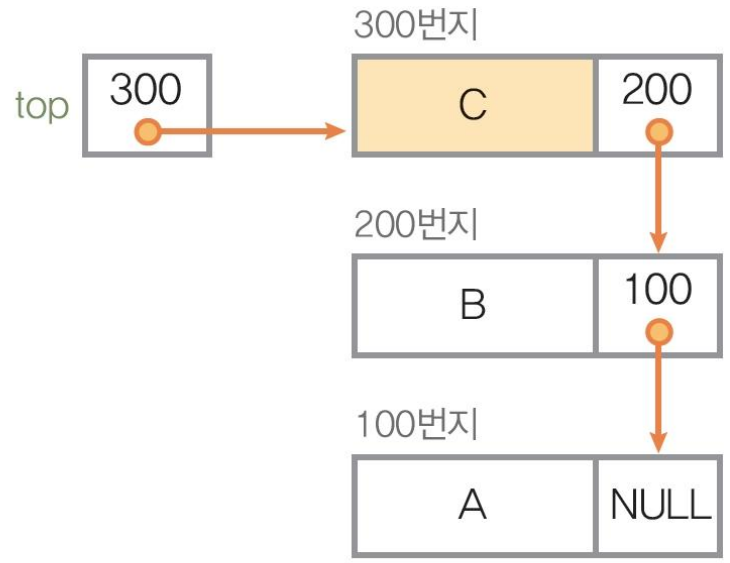
2 원소 A 삽입 : push(stack, A);



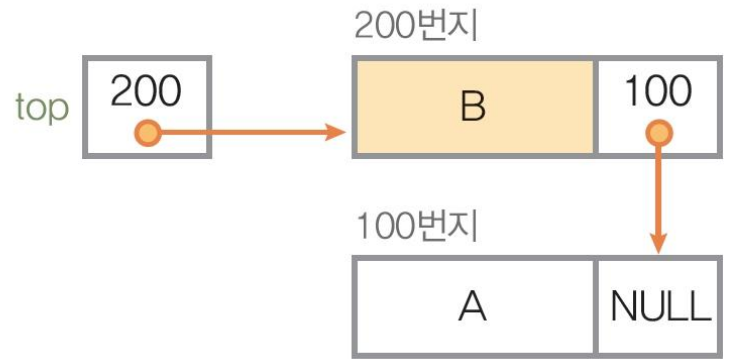
3 원소 B 삽입 : push(stack, B);



4 원소 C 삽입 : push(stack, C);



5 원소 삭제 : pop(stack);



- [예제 5-2] 연결 자료구조를 이용해 연결 스택 구현하기 : [교재 251p](#)
- 실행 결과

**** 연결 스택 연산 ****

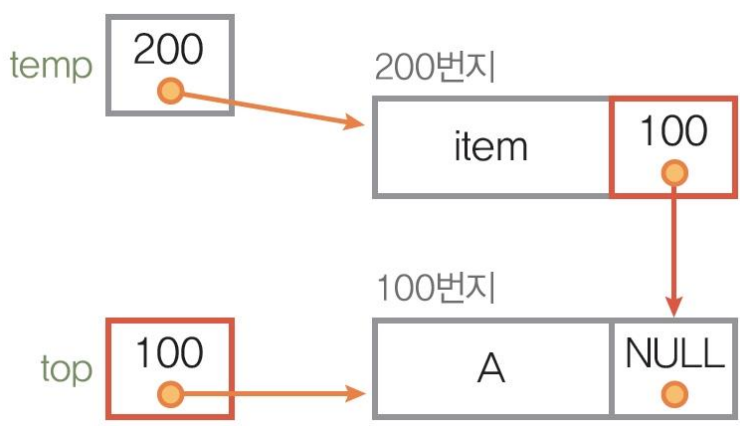
```
STACK [ ]  
STACK [ 1 ]  
STACK [ 2 1 ]  
STACK [ 3 2 1 ]  
STACK [ 3 2 1 ]    peek => 3  
STACK [ 2 1 ]      pop  => 3  
STACK [ 1 ]        pop  => 2  
STACK [ ]          pop  => 1
```

[stackL.c]

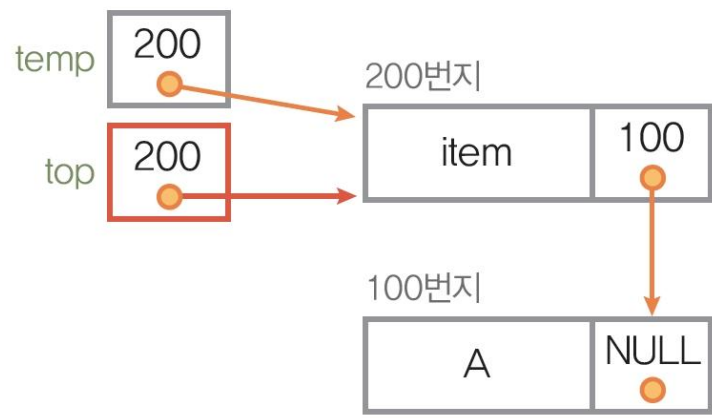
- 11~17행 : 연결 스택에서 삽입 연산을 수행하는 과정
 - 12행 : 원소 item을 저장할 노드에 대한 메모리 할당, 포인터 temp 설정
 - 14행 : 삽입할 노드의 데이터 필드에 원소 item을 저장



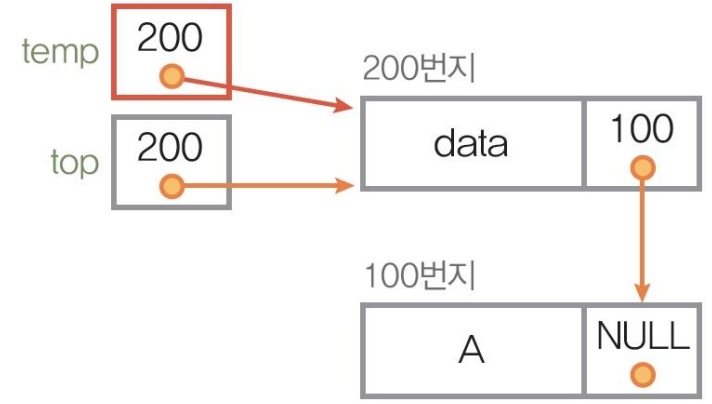
- 15행 : 삽입할 노드의 링크 필드에 포인터 top의 값 저장하면, 새로 삽입한 노드가 현재 스택의 마지막 노드(현재 top이 가리키는 노드)로 연결



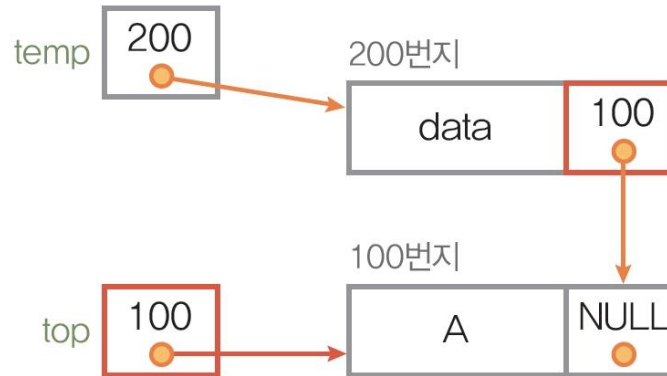
- 16행 : 포인터 temp의 값(삽입 노드의 주소)을 포인터 top에 설정, 새로 삽입한 노드가 스택의 top 노드가 되도록 조정



- 20~34행 : 연결 스택에서 삭제하는 연산을 수행하는 과정
 - 22행 : 포인터 temp를 top 노드에 설정하여 삭제할 노드를 가리킴



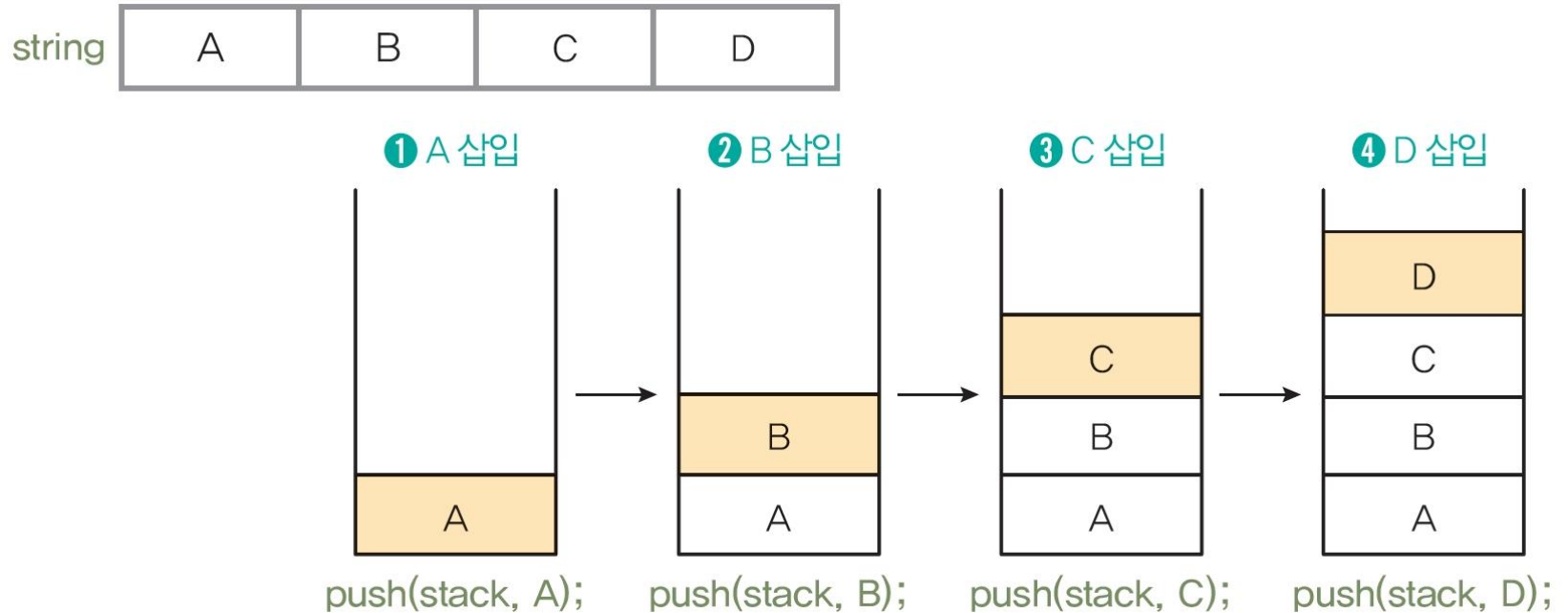
- 29행 : 스택의 마지막 노드의 데이터 필드값을 변수 item에 저장
- 30행 : 포인터 top의 위치를 현재 마지막 노드의 아래 노드로 이동



- 31행 : 포인터 temp가 가리키는 노드를 메모리 해제
- 32행 : 변수 item의 값, 즉 스택의 top이었던 노드의 데이터를 반환
- 51~54행 : 연결 스택에 노드가 있는 동안, top 노드부터 링크 필드를 따라 아래 노드로 이동하면서 데이터 필드값을 출력. top 노드부터 출력하므로 출력 화면에서 왼쪽이 스택의 마지막 원소인 top이 됨

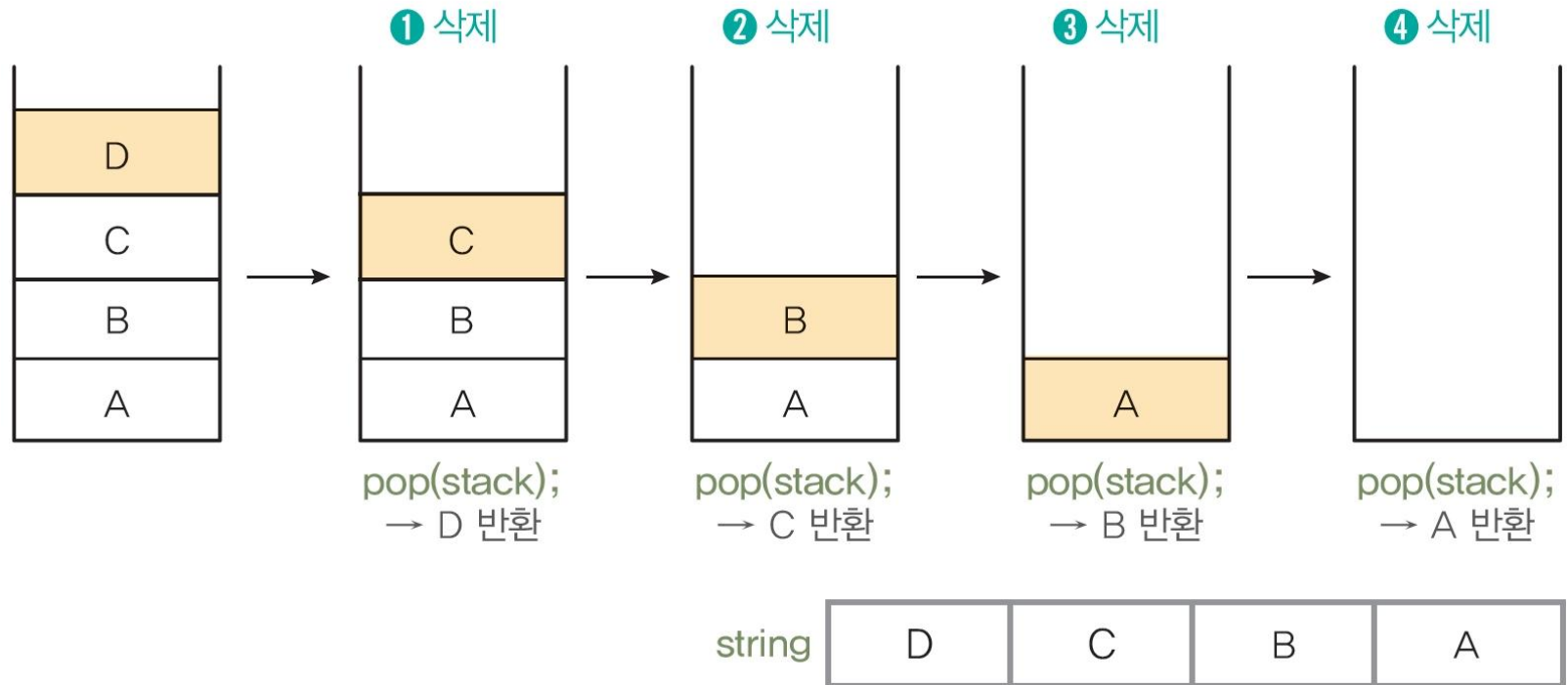
3. 스택의 응용 : 스택을 이용한 역순 문자열

- 역순 문자열 만들기
 - 스택의 후입선출(LIFO) 성질을 이용
 - ① 문자열을 순서대로 스택에 삽입



3. 스택의 응용 : 스택을 이용한 역순 문자열

② 스택에서 삭제하여 문자열 만들기



3. 스택의 응용 : 스택을 이용한 역순 문자열

■ 역순 문자열

– 문자열을 입력받아 스택을 이용하여 거꾸로 출력하는 프로그램을 작성하시오.

1. 문자열을 한 글자씩 스택에 push
2. 스택에서 pop하면서 출력

Initialize empty stack

Input string S

1. 문자열을 스택에 넣기 (push)

for each character c in S:

push c into stack

2. 스택에서 꺼내면서 출력 (pop)

while stack is not empty:

c ← pop from stack

print c

3. 스택의 응용 : 스택을 이용한 수식의 괄호 검사

■ 수식의 괄호 검사

- 수식에 포함되어있는 괄호는 가장 마지막에 열린 괄호를 가장 먼저 닫아 주어야 하는 후입선출 구조
- 후입선출 구조의 스택을 이용하여 괄호를 검사
- 수식을 왼쪽에서 오른쪽으로 하나씩 읽으면서 괄호 검사

① 왼쪽 괄호를 만나면 스택에 **push**

② 오른쪽 괄호를 만나면 스택을 **pop**하여 마지막에 저장한 괄호와 같은 종류인지를 확인

- 같은 종류의 괄호가 아닌 경우 괄호의 짝이 잘못 사용된 수식임.

- 수식에 대한 검사가 모두 끝났을 때 스택은 공백 스택이 됨
 - 수식이 끝났어도 스택이 공백이 되지 않으면 괄호의 개수가 틀린 수식임.

3. 스택의 응용 : 스택을 이용한 수식의 괄호 검사

알고리즘 5-3 수식의 괄호 쌍 검사

```
testPair()
  exp ← Expression;  ← 수식 Expression 을 읽어서 exp 에 저장
  Stack ← null;
  while (true) do {
    symbol ← getSymbol(exp);  ← 수식 exp 에서 문자 1개를 읽어 들임
    case {
      symbol == "(" or "[" or "{" :
        push(Stack, symbol);
      symbol == ")" :
        open_pair ← pop(Stack);
        if (open_pair ≠ "(") then return false;
      symbol == "]" :
        open_pair ← pop(Stack);
        if (open_pair ≠ "[") then return false;
      symbol == "}" :
        open_pair ← pop(Stack);
        if (open_pair ≠ "{") then return false;
      symbol == null :
        if (isEmpty(Stack)) then return true;
        else return false;
    }
  }
end testPair()
```

3. 스택의 응용 : 스택을 이용한 수식의 괄호 검사

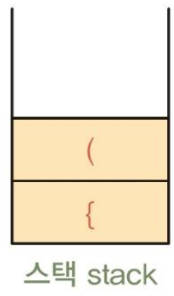
- 예

{(A+B)-3}*5+[{cos(x+y)+7}-1]*4

{ (A + B) - 3 } * 5 + [{ cos (x + y) + 7 } - 1] * 4

① push(stack, {)

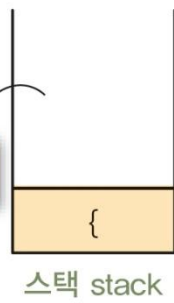
② push(stack, ()



{ (A + B) - 3 } * 5 + [{ cos (x + y) + 7 } - 1] * 4

③ pop(stack)

← 같은 종류인지 확인 → 삭제된 원소 : (



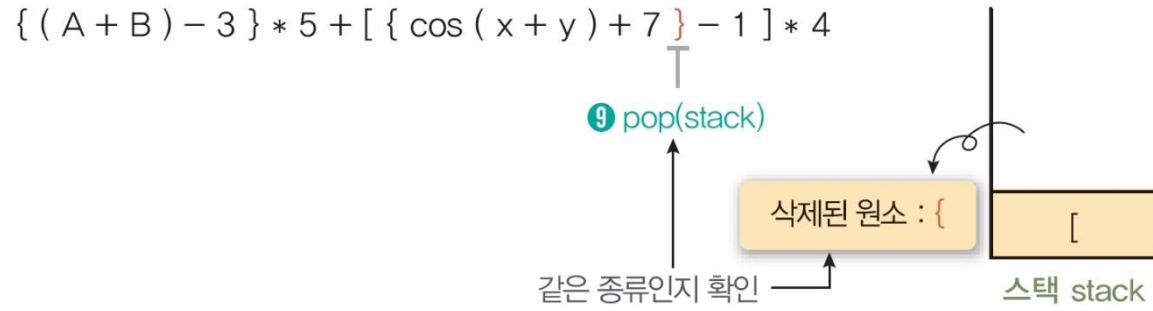
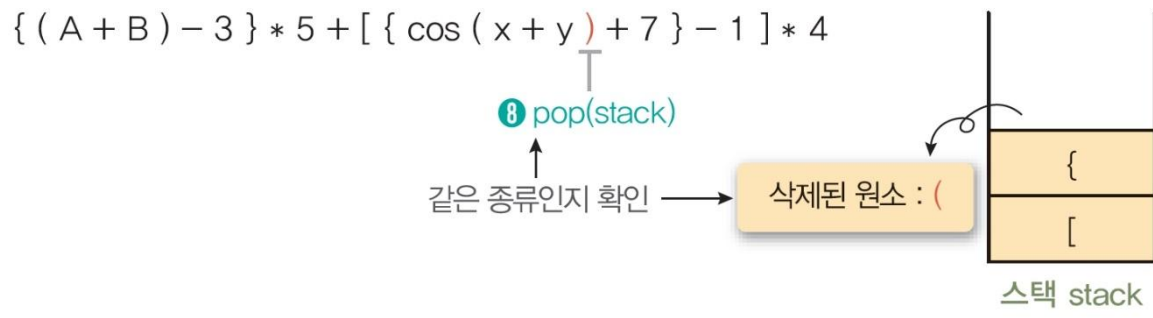
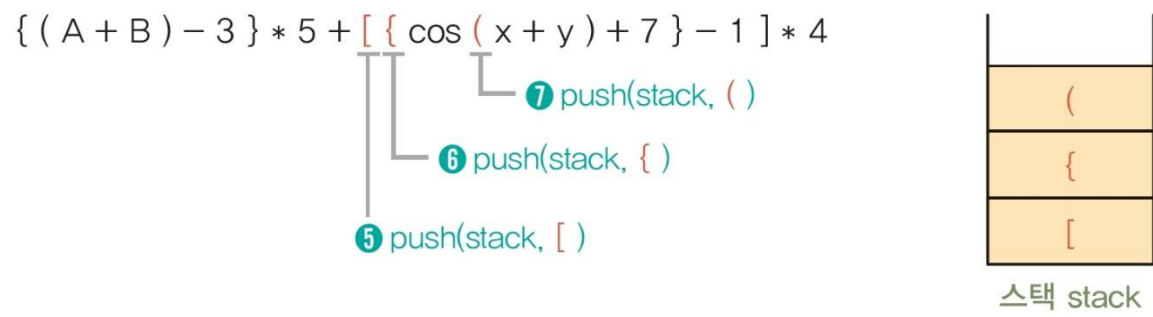
{ (A + B) - 3 } * 5 + [{ cos (x + y) + 7 } - 1] * 4

④ pop(stack)

← 같은 종류인지 확인 → 삭제된 원소 : {



3. 스택의 응용 : 스택을 이용한 수식의 괄호 검사



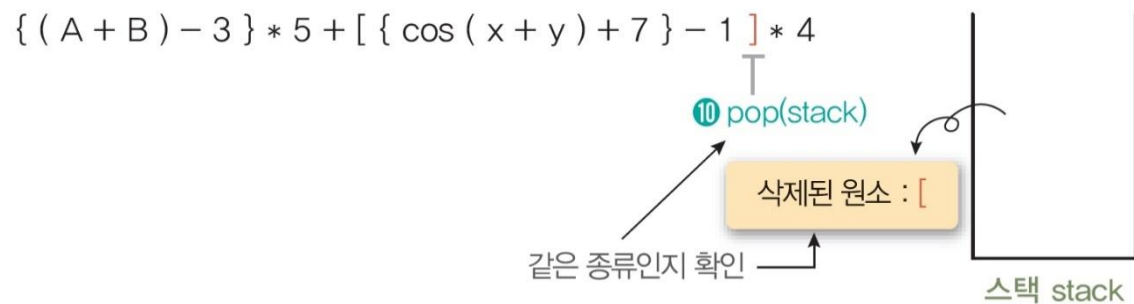
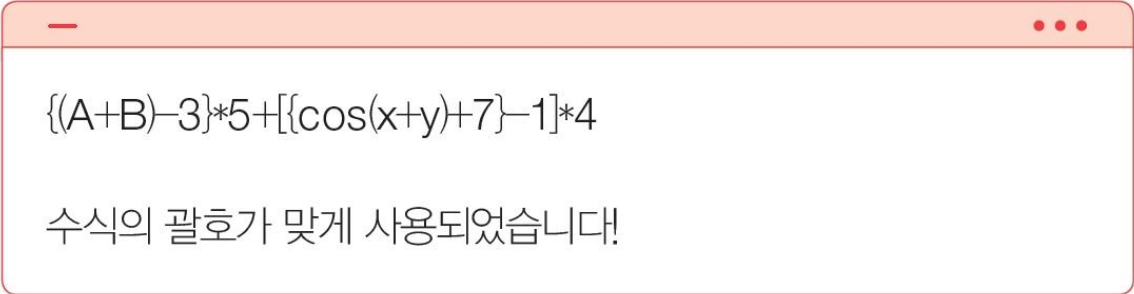


그림 5-10 수식 처리에 따른 스택 사용 과정

3. 스택의 응용 : 스택을 이용한 수식의 괄호 검사

- [예제 5-3] 스택을 이용해 수식의 괄호쌍 검사하기 : [교재 261p](#)
- 실행 결과



$\{(A+B)-3\}*5+[\{\cos(x+y)+7\}-1]*4$

수식의 괄호가 맞게 사용되었습니다!

- 코드 구성:
 - Stack 코드
 - Stack 된 코드가 pair를 이루고 있는지 확인하는 코드
 - Main 코드